

INFORMAÇÕES SOBRE A AVALIAÇÃO

AV1	AVALIAÇÃO 1 - 25 pontos
-----	-------------------------

INFORMAÇÕES DOCENTE						
CURSO:	DISCIPLINA:	TURNO	MANHÃ	TARDE	NOITE	PERÍODO/SALA:
ENGENHARIA DE SOFTWARE	DIAW		x	x	x	2º
PROFESSOR (A): João Paulo Carneiro Aramuni						

INFORMAÇÕES DISCENTE	
ALUNO(A):	RA:
DATA:	NOTA:
INSTRUÇÕES: <u>Consulta:</u> - Será permitido consultar os seus próprios projetos desenvolvidos em laboratório. - Será permitido consultar a documentação oficial do Spring Boot (spring.io) e do W3C (w3.org) para HTML e CSS. - Não será permitido o uso de inteligência artificial, como o ChatGPT, Claude AI, Gemini, GitHub Copilot, etc., bem como consultas ao Google ou a qualquer outro site de busca. <u>Tecnologias:</u> Será obrigatório o uso de Java 25, Spring Boot, Apache Maven, Thymeleaf, HTML e CSS. <u>Aparelhos eletrônicos:</u> Não será permitido o uso de celulares, relógios digitais ou qualquer outro aparelho eletrônico durante a realização da prova. <u>Conduta:</u> O aluno que, durante a prova, utilizar meios ilícitos, como comunicação com outros alunos, consulta a qualquer tipo de material extra prova ou uso de aparelhos eletrônicos, será eliminado do processo avaliativo e receberá nota 0 (zero). <u>Funcionamento:</u> Projetos que não rodam, ou seja, que não compilam, ou que não possam ser acessados via http://localhost:8080/ serão avaliados com até no máximo 50% da nota da prova (12,5 pts). Boa Prova!	

Sistema de Consulta de Candidatos TSE

[10,0 pts] Questão 1) Construa o arquivo **controller/CandidatosTseController.java** e realize as chamadas necessárias para os arquivos disponibilizados na prova, chamados **service/CandidatosTseService.java** e **model/Candidato.java**.

Dicas:

- O Controller é uma classe anotada com `@Controller` (não `@RestController`, pois ele retorna uma view do Thymeleaf, não JSON).
- Injete o `CandidatosTseService` via construtor:

```
@Controller
public class CandidatosTseController {

    private final CandidatosTseService candidatosTseService;

    public CandidatosTseController(CandidatosTseService candidatosTseService) {
        this.candidatosTseService = candidatosTseService;
    }
    // ...
}
```

- Crie um único endpoint (GET /) que atenda tanto o carregamento inicial da página quanto os filtros; os filtros são simplesmente parâmetros de query string (?genero=...&escolaridade=...&idadeMin=...&idadeMax=...).
- Use @RequestParam(required = false) para cada filtro, já que o usuário pode não preencher nenhum. Atenção: idadeMin e idadeMax são números, então use o tipo Integer (e não int), que aceita null quando o campo fica vazio:

```
@GetMapping("/")
public String index(
    @RequestParam(required = false) String genero,
    @RequestParam(required = false) String escolaridade,
    @RequestParam(required = false) Integer idadeMin,
    @RequestParam(required = false) Integer idadeMax,
    Model model) {
    // ...
}
```

- Confira a assinatura exata do método filtrarPerfil(...) no CandidatosTseService fornecido; os parâmetros e a ordem precisam bater certinho.
- Você vai precisar passar pro Model (para o Thymeleaf usar no HTML):
- a lista filtrada de candidatos;
- o total de candidatos encontrados (útil pra mensagem "X candidato(s) encontrado(s)" e pra saber quando mostrar "nenhum resultado");
- as opções de filtro (lista de gêneros e de escolaridades distintos), o service já deve ter métodos prontos para isso (algo como listarGeneros() e listarEscolaridades());
- os valores atualmente selecionados de cada filtro, para que o formulário "lembre" o que o usuário marcou/selecionou/digitou depois de recarregar a página.
- Cuidado com null: se genero ou escolaridade vierem null, ao devolver pro HTML prefira transformar em string vazia (""), assim evita null aparecendo escrito na tela ou quebrando comparações no Thymeleaf. Já idadeMin e idadeMax podem ser devolvidos como vieram (null deixa o campo numérico vazio).
- O método deve retornar o nome da view (uma String), não o objeto HTML, o Thymeleaf resolve isso a partir do nome (ex.: return "index"; aponta pra templates/index.html).

[10,0 pts] Questão 2) Construa o arquivo **templates/index.html**

Dicas:

Estrutura geral da página

- Declare o namespace do Thymeleaf no <html>:
 - <html lang="pt-BR" xmlns:th="http://www.thymeleaf.org">
- Referencie o CSS com th:href (não href puro), para que o Thymeleaf resolva o caminho junto com o contexto da aplicação:
 - <link rel="stylesheet" th:href="@{/css/style.css}">

O formulário de filtro

- Use `method="get"` — assim os filtros viram parâmetros de URL, sem precisar de JavaScript nem de outro endpoint:
 - `<form method="get" th:action="@{/}">`
- Para os campos de idade, use inputs do tipo `number` com `th:value`, para manter o que o usuário digitou depois de filtrar:
 - `<input type="number" name="idadeMin" th:value="${idadeMin}">`
 - `<input type="number" name="idadeMax" th:value="${idadeMax}">`
- Para o gênero, use botões de opção (`radio`), todos com o mesmo `name`. Você vai precisar de uma opção "Todos" (valor vazio) e das opções dinâmicas vindas da lista do Model, usando `th:each`. Para marcar a opção escolhida, use `th:checked` comparando o valor da opção com o valor atualmente selecionado:

```
<label>
  <input type="radio" name="genero" value="" th:checked="${generoSelecioneado == ''}">
  Todos
</label>
<label th:each="g : ${generos}">
  <input type="radio" name="genero" th:value="${g}" th:checked="${g == generoSelecioneado}">
  <span th:text="${g}">Gênero</span>
</label>
```

- Para o `<select>` de escolaridade, a lógica é a mesma: uma opção "Todas" (valor vazio) e as opções dinâmicas com `th:each`. Preste atenção em marcar qual opção deve ficar selecionada por padrão, comparando o valor da opção com o valor atualmente selecionado:

```
<select name="escolaridade">
  <option value="" th:selected="${escolaridadeSelecioneada == ''}">Todas</option>
  <option th:each="e : ${escolaridades}"
    th:value="${e}"
    th:text="${e}"
    th:selected="${e == escolaridadeSelecioneada}"></option>
</select>
```

A listagem de candidatos

- Use `th:each` num `<article>` (ou outro elemento) pra iterar a lista de candidatos vinda do Model:
 - `<article class="card-candidato" th:each="cand : ${candidatos}">`
- Pra foto, monte a URL concatenando o caminho da pasta com o nome do arquivo (o Candidato já deve ter um método pronto pra isso, ex.: `getNomeArquivoFoto()`):
 - ``

- Use `th:text` para exibir os campos do candidato (nome de urna, número, cargo, partido, etc.). Você pode juntar dois campos numa mesma linha concatenando dentro da expressão:
 - `<p th:text="{cand.cargo + ' · ' + cand.siglaPartido}">Cargo · Partido</p>`
- Para a etiqueta de gênero, use `th:classappend` para acrescentar uma classe CSS diferente conforme o valor (etiqueta-f para FEMININO, etiqueta-m para MASCULINO). Assim o CSS pinta cada uma de uma cor:

```
<span class="etiqueta"
  th:classappend="{cand.genero == 'FEMININO' ? 'etiqueta-f' : (cand.genero == 'MASCULINO'
? 'etiqueta-m' : '')}"
  th:text="{cand.genero}">Gênero</span>
```

- Esconda com `th:if` os campos que às vezes vêm vazios ou sem valor útil (idade, escolaridade, ocupação):

```
<span class="etiqueta etiqueta-idade" th:if="{cand.idade > 0}"
  th:text="{cand.idade + ' anos'}">00 anos</span>

<p th:if="{cand.grauInstrucao != null and cand.grauInstrucao != '#NULO' and
!cand.grauInstrucao.isBlank()}"
  th:text="{cand.grauInstrucao}">Escolaridade</p>

<p th:if="{cand.ocupacao != null and cand.ocupacao != '#NULO' and !cand.ocupacao.isBlank()}"
  th:text="{cand.ocupacao}">Ocupação</p>
```

- Contagem de resultados e lista vazia:

```
<p class="resultado-info">
  <strong th:text="{totalEncontrado}">0</strong> candidato(s) encontrado(s).
</p>

<p th:if="{totalEncontrado == 0}" class="sem-resultado">
  Nenhum candidato encontrado com esses filtros.
</p>
```

- Os botões do formulário:
 - `<button type="submit">Filtrar</button>`
 - `<a th:href="@{/}" class="limpar">Limpar</a>`

[5,0 pts] Questão 3) Construa o arquivo `static/css/style.css`

Dicas:

- Use CSS Grid para dividir a página em duas colunas: o painel de filtros com largura fixa à esquerda e a lista de candidatos ocupando o restante do espaço:
 - ```
.layout {
 display: grid;
 grid-template-columns: 260px 1fr;
 gap: 20px;
}
```

- Use CSS Grid também na lista de cards, de forma responsiva, sem precisar definir manualmente quantas colunas cabem em cada tamanho de tela:
  - `.lista-candidatos {  
display: grid;  
grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));  
gap: 12px;  
}`
- Cada card é horizontal (foto à esquerda, dados à direita). Use Flexbox no card e dê à foto uma largura fixa, padronizando a proporção com aspect-ratio para evitar fotos esticadas ou cortadas de forma estranha:
  - `.card-candidato {  
display: flex;  
gap: 12px;  
}`
  - `.foto-candidato {  
width: 90px;  
aspect-ratio: 161 / 225;  
object-fit: cover;  
}`
- Crie as classes das etiquetas usadas no th:classappend, com uma cor para cada gênero:
  - `.etiqueta-f {  
background-color:  
#f7e1f0;  
color:  
#a0226f;  
}`
  - `.etiqueta-m {  
background-color:  
#e1ecf7;  
color:  
#1f5c99;  
}`
- Use uma media query para que, em telas estreitas, o painel de filtros fique acima da lista, em uma única coluna:
  - `@media (max-width: 800px) {  
layout {  
grid-template-columns: 1fr;  
}  
}`

Observações:

- Não se esqueça de copiar o arquivo data/candidatos/consulta\_cand\_2026\_MG.csv para a pasta resources do projeto.
- Copie também a pasta images/candidatos para a pasta static do projeto.
- Leia o arquivo README.md antes de começar a prova.